

Chapter 29 CAN

29.1 Overview

CAN (Controller Area Network) bus is a robust vehicle bus standard designed to allow microcontrollers and devices to communicate with each other in applications without a host computer. It is a message-based protocol, designed originally for multiplex electrical wiring within automobiles to save on copper, but is also used in many other contexts.

29.2 Features

The features listed below which may or may not be present in actual product, may be subject to the third party licensing requirements. Please contact Rockchip for actual product feature configurations and licensing requirements.

CAN controller supports the following features:

- Support CAN 2.0B protocol
- Support 32-bit APB bus
- Support transmit or receive CAN standard frame
- Support transmit or receive CAN extended frame
- Support transmit or receive data frame, remote frame, overload frame, error frame and frame interval
- Support transmit or receive error count
- Support acceptance filtering
- Support Bit error, Bit stuffing error, Form error, ACK error and CRC error
- Support multiple types of interrupt and all interrupt can be masked
- Support CAN controller status query
- Support capture the bit position of arbitration section where the arbitration was lost
- Support error code check
- Support self-test mode
- Support single sample and three sample configurable
- Support SJW(Resynchronization Jump Width) configurable
- Support BRP(system prescale coefficient) configurable
- Support bit timing configurable
- Support loop-back mode for self-test operation
- Support silent mode for debug
- Support receive self-transmit data mode(rxstx_mode)
- Support TX data and RX data order select
- Support RX data cover mode
- Support auto retransmission mode
- Support auto bus on after bus-off state
- Support space_rx_mode
- Support transmitter delay compensation and SSP position configurable
- Support sleep mode
- Support timestamp
- Support transmit event FIFO
- Support 2 transmit buffers
- Support receive buffer/FIFO mode
- Support protocol exception event
- Support DMA

29.3 Function Description

29.3.1 Block Diagram

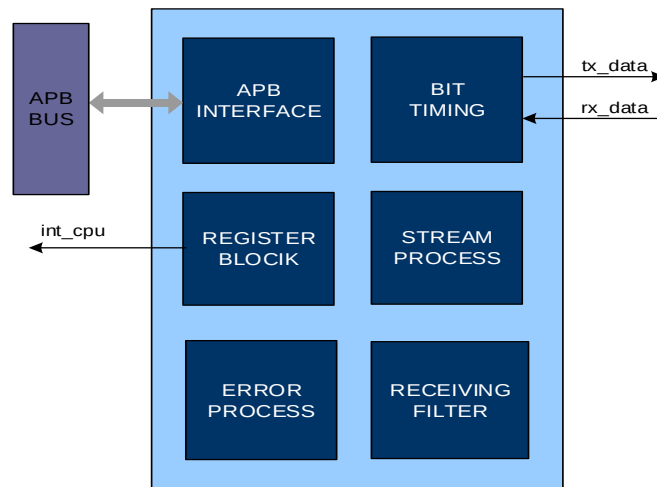


Fig. 29-1 CAN Controller Block Diagram

This version of CAN controller includes the previous version of CAN logic in compliance with old software application and new version of CAN logic with new software application.

29.3.2 ACCEPTANCE FILTER

The acceptance filter performs filtering using the acceptance ID register and the ID mask register. The acceptance filter uses multiple-time filtering. It includes the old version receiving filter and five addition ID filters. Each ID filter pair has a Filter Mask register and a Filter ID register. Each filter pair is controlled by corresponding FILTER CTRL bit in AFR register. For each pair using ID register and ID mask register (controlled by acceptance filter register) sampling all the bits of the ID, it is compared with the ID register. It is not a comparison every time a bit is sampled. And don't check the bit in the ID mask register that is 1. Once all the ID bits have passed the filtering, the controller considers the frame data as the desired ID, thereby performing the next operation.

29.3.3 BIT TIMING

29.3.3.1 Bit Timing Logic

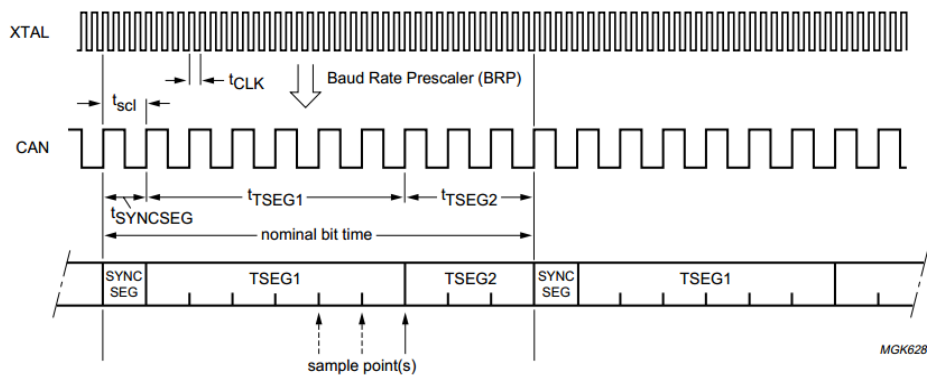
The bit timing logic controls the sampling point position through the buffer bits in the timing register to ensure the accuracy of the data sampling. The bit timing logic receives the clock frequency division signal for identification, sets the bus timing parameters, establishes synchronization parameters, and adjusts the bus transmission rate. At the same time, the bus is monitored and the message to be sent is transmitted to the bus at the set timing. According to the provisions of the CAN protocol: the CAN bus is always at a high level when no message is sent, and the continuous recessive bit is monitored on the bus. At this time, the bus is in an idle state, and the arbitration priority is low, ready to receive data at any time. When the bus detects a transition from a recessive bit to a dominant bit, it is proved that the frame start bit starts transmitting, and the bus performs a hard synchronization in the bit start sync segment. Then, in the process of receiving the message, once the edge of the transition close to the sampling point is detected and the edge of the edge and the synchronization segment have a phase error lower than the synchronization width (SJW is taken when the value exceeds SJW), the controller executes once resynchronization, resynchronization can be performed multiple times during one data transmission.

29.3.3.2 Bit Timing Definition

The transmission time of each bit is divided into 4 parts:

$$t_{nbt} = t_{sync_seg} + t_{prop_seg} + t_{phase_seg1} + t_{phase_seg2}$$

t_{sync_seg} , t_{prop_seg} , t_{phase_seg1} and t_{phase_seg2} are integer multiples of the unit time(t_{sclk}), and



Possible values are BRP = 000001, TSEG1 = 0101 and TSEG2 = 010.

Fig. 29-4 Three sampling diagram

Sending Point: The location of the transmit point should be at the beginning of each bit time according to the protocol, and it is designed to synchronize it with the go_sync signal. In addition, if it is in sync or resynchronize, transmit point will be valid immediately.

29.3.3.4 Bit Synchronization

With the synchronization method, one or more nodes that satisfy the synchronization condition align their synchronization segments with the transmission data on the bus at a specific time. Synchronization occurs on the 1-to-0 transition edge in order to control the distance from the transition edge to the sample point. Two synchronization methods are defined in the CAN bus communication protocol: hard synchronization and resynchronization.

Hard sync: All nodes must be synchronized to the leading edge of the starting frame of the node that first started transmitting the message. At the beginning of each frame of data, a synchronization action is taken between the nodes.

Hard synchronization implementation method: determine whether the bus state meets the frame start condition specified in the protocol, and the node is not in the state of the data to be transmitted. At this time, a hard synchronization flag signal hard sync is generated, and the pulse width is a system clock; The hard sync entry is added to the control condition of the go_seg1 signal, and go_seg1 is valid immediately when the hard sync signal is '1'. Therefore, once the hard synchronization condition is met, the system enters the phase_seg1 segment, thereby achieving synchronization.

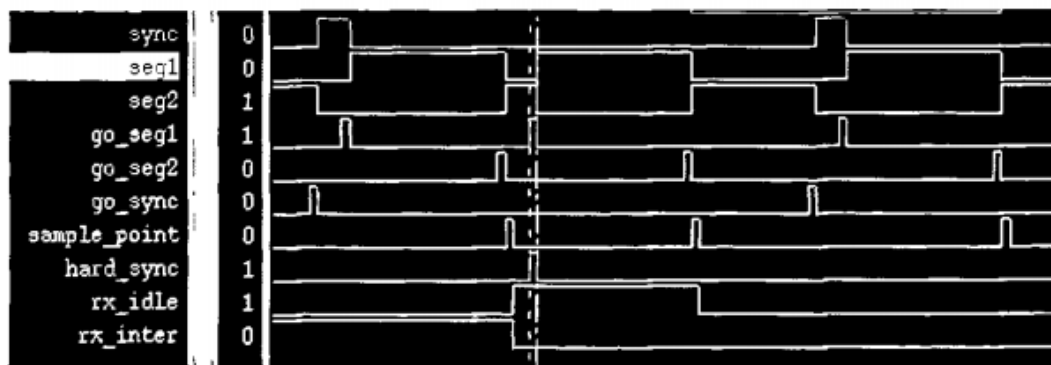


Fig. 29-5 Hard sync waveform diagram

Resynchronization: In addition to generating hard synchronization at the beginning of each frame of data, the CAN bus communication protocol also specifies the transmission of data for each frame. When the timing coordination between the nodes is not ideal, it will be resynchronized, so that the cooperation between the nodes is in a good state.

For the receiver, the bus changes it received should occur in the sync segment. Once the receiver's status violates this rule, the receiver will perform a resynchronization.

Two situations that require resynchronization:

- The jump of the bus value '1' to '0' occurs between the sync segment and the sample

- point, which is a delay jump;
- The jump of the bus value '1' to '0' occurs between the sample point and the sync segment, which is an early jump.

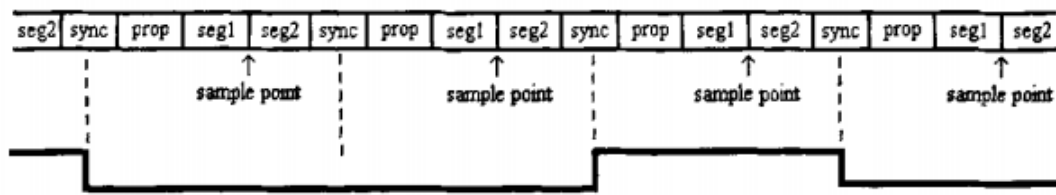


Fig. 29-6 Resynchronization

Resynchronization method:

1. Condition for resynchronization: the receiver is in a mode of receiving data (not inter-frame space or bus idle), and a resynchronization flag signal 'resync' is generated when a jump of the bus value '1' to '0' occurs;
2. According to the position of the resynchronization flag signal, it is determined that the abnormal jump is a delay jump or an early jump;
3. Counting the delay or the length of the advance by the counter;
4. In this bit, phase_seg1 is added with a delay time or phase_seg2 is subtracted from the advance time to obtain a new phase buffer time for the bit, thereby achieving synchronization.

29.3.4 STREAM PROCESS

29.3.4.1 Data Buffering

The data cache includes two parts: a transmit buffer and a receive buffer. The data to be sent on the CAN bus is loaded into the buffer area. This buffer area is called the "transmission buffer"; The data received from the CAN bus is also stored in the buffer area, which is called the "receiver buffer". The size of the buffer is 13 bytes, including one-byte frame information, 2 to 4 bytes identifiers (standard or extended frame), and 8 bytes of data.

CPU configures the transmission of the data information (frame information, identification code, data), and then enable the transmit bit. The transmit data information is spliced into a parallel complete data to be transmitted (including the CRC check code) according to the current frame information and stored in transmission buffer. Stream process unit wait for the appropriate time to send data to the bit timing unit.

The received message is placed in the receiving buffer. After receiving, the receiving completion interrupt will be generated. After receiving the interrupt, the CPU will read the data in the receiving buffer and clear the receiving buffer, and STATE [0] will be 0. If the CPU does not read in time, STATE [1] will remain at 1. If a new message is received at this time, overflow will occur and not respond with ack by default.

In new design, there is another independent transmit buffer which has another corresponding transmit request bit. Besides, there is a receive FIFO can be configured to use. Rx data has a RDATA address to be read by host. The size of data is 18 word (one-word frame information, one-word Id, 16-words data) for transmit and 19 word (1-word timestamp additional) for receive. Rx FIFO has 32bits×128 memory space which allow to contain 6 receive message.

Two transmit request bit in CAN_CMD corresponds to two transmit buffers. Transmit message must written to registers before setting transmit request bit. After setting transmit request bit, transmit messages in registers stores in corresponding transmit buffer. When the RKCAN buffer1 transmission frame complete, 'tx_req_1' is automatically cleared to 0. CAN support RX FIFO. When FIFO enabled, receive data stored in FIFO as Receive FIFO message format shown below. Receive data should read from the RXFIFO entrance. IF FIFO is disabled, receive data stored in RXDATA registers and should read from those registers every time.

Table 29-1 Receive FIFO message format

NAME	Offset	Access	Description
RX FIFO RB0-INFO	0x0000	RO	Same as <u>CAN_RXFRAMEINFO</u>
RX FIFO RB0-ID	0x0004	RO	Same as <u>CAN_RXID</u>
RX FIFO TB0-TIMESTAMP	0x0008	RO	RX TIMESTAMP VALUE
RX FIFO RB0-DATA0	0x000c	RO	Same as <u>CAN_RXDATA</u>
RX FIFO RB0-DATA1	0x0010	RO	Same as <u>CAN_RXDATA</u>
RX FIFO RB0-DATA2	0x0014	RO	Same as <u>CAN_RXDATA</u>
...	...	...	...
RX FIFO RB0-DATA15	0x0048	RO	Same as <u>CAN_RXDATA</u>

29.3.4.2 Receive Data

According to the bus protocol, taking the data frame of the extended frame format as an example, the receiving data state machine is as follows:

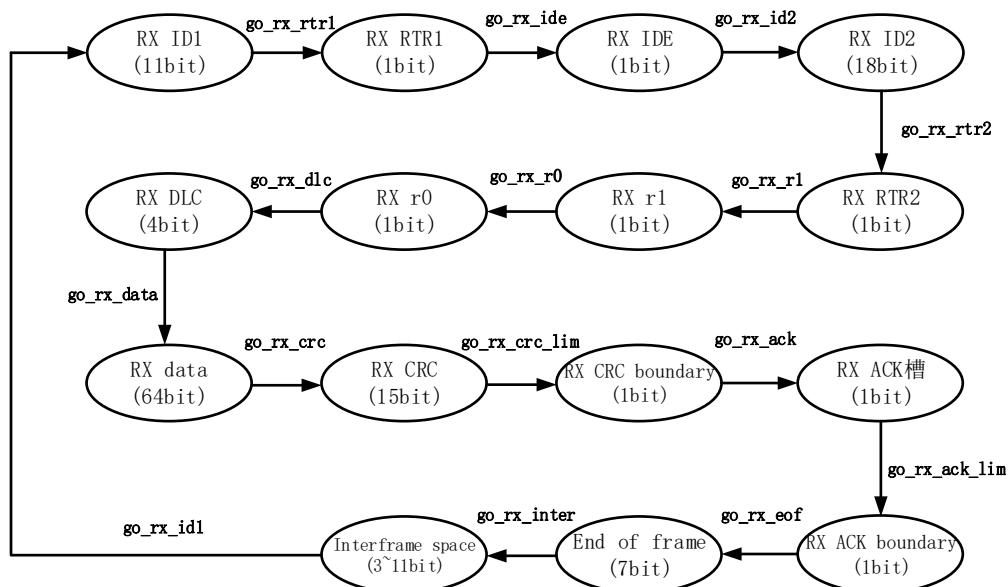


Fig. 29-7 Receive data state diagram

Each state of the state machine corresponds to which "field" of the frame data the receiver is in when receiving data, and only one of all state flags is valid at the same time. The method of implementation is as follows: Define a bit counter, add one to each sample point (provided by the bit timing module). At the same time, according to the current state of the state machine, it is determined in this state that the bit counter needs to count to what value (determined by the length of each field defined in the protocol) to generate a jump signal to the next state transition. Whenever a hopping signal is generated, the counter is cleared and ready to begin counting for the next state.

The receiving state machine ensures that the bus controller can automatically recognize the current receiving state when receiving data. After that, the data of each data field needs to be stored in the corresponding register according to different states, and then the next operation is performed.

29.3.4.3 Transmit Data

CAN accesses message element space of a buffer in TX block only if the respective request bit is set. The Host must respect access rules to avoid memory collisions, that is, after the Host sets a buffer ready request through the CMD register, it should not read or write the respective message element space until the respective request bit is in a clear/unset state.

Parallel data is serially transmitted to the bus according to a specific timing. The sending point is the tx_point. The transmit counter is self-added in the valid data portion of the data frame, and is cleared when one frame of data is transmitted or the bus is in an error state. The implementation is as follows: Combine all the data to be sent into a completely parallel data chain according to the currently set frame type; Using tx_pointer as a data link pointer, obtaining a serial data signal (tx_bit) to be transmitted; Determine what type of data to send according to the current state, and determine the data tx_next to be sent at the next transmission time; Synchronize the data transmission signal tx with the tx_next through the transmission point tx_point to obtain a true transmission signal tx.

29.3.4.4 Bit Stuffing

Bit stuffing is a function set to prevent burst errors. Add a bit of inversion data when the same level lasts 5 bits.

● Sending unit

The data between the SOF and CRC segments when transmitting data frames and remote frames. If the same level lasts for 5 bits, the next bit (bit 6) is inserted with the level of the 1st bit and the first 5 bits.

● Receiving unit

Data between SOF and CRC segments when receiving data frames and remote frames. If the same level lasts for 5 digits, the next bit (bit 6) needs to be deleted and received. If the 6th bit is the same level as the first 5 bits, it will be treated as an error and an error frame will be sent.

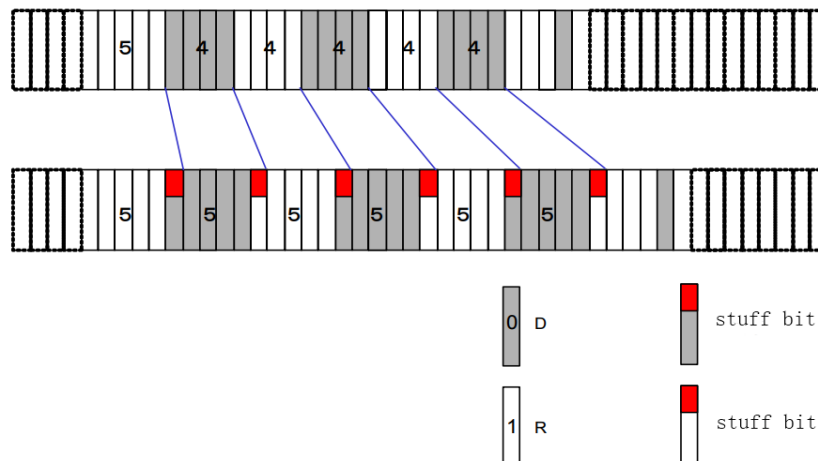


Fig. 29-8 Bit Stuffing

29.3.4.5 Transmit Event FIFO

The Transmit Event FIFO (TEF) allows the application to keep track of the order and time the messages were transmitted. The TEF works similar to a Receive FIFO, which is a 32bits×16 FIFO. Instead of storing received messages, it stores transmitted messages. Messages are only saved if TEF enable is set. The Sequence Number (SEQ) of the transmitted message is copied into the TEF Object. The payload data are not stored. Transmitted messages are timestamped if TEFTSEN is set. Transmit message format store in TXE FIFO as follow:

Table 29-2 TXE FIFO message format

NAME	Offset	Access	Description
TXE FIFO TB0-INFO	0x0000	RO	Same as <u>CAN_TXFRAMEINFO</u>
TXE FIFO TB0-ID	0x0004	RO	Same as <u>CAN_TXID</u>
TXE FIFO TB0-TIMESTAMP	0x0008	RO	TX TIMESTAMP VALUE
TXE FIFO TB1-INFO	0x000c	RO	Same as <u>CAN_TXFRAMEINFO</u>
TXE FIFO TB1-ID	0x0010	RO	Same as <u>CAN_TXID</u>
TXE FIFO TB1-TIMESTAMP	0x0014	RO	TX TIMESTAMP VALUE
...	...	...	...

29.3.4.6 TIMESTAMP

CAN Controller module contains a Time Base Counter(TBC). The TBC is a 32-bit free-running counter that increments on multiples of CAN clock and rolls over to zero. TBC can be cleared by writing any value to TBC.

- **TIMESTAMP_CTRL** is used to configure the prescale for the TBC.
- Setting **TIMESTAMP_CTRL.TBCEN** enables the TBC.
- Clearing **TBCEN** disables, stops and resets the TBC.
- The TBC has to be disabled before writing to TBC by clearing **TBCEN**.
- The application can read TBC at any time. Like with any multibyte counter, the application has to consider that the counter increments and might rollover between reading the different bytes of the counter. A rollover of the TBC will generate an interrupt.

Capturing of time stamps at the sample points of the Start of Frame bits of received and transmitted frames.

29.3.5 ERROR PROCESS

There are five types of errors. Multiple errors can occur at the same time.

● **BIT ERROR:**

A unit that is sending a bit on the bus also monitors the bus. A BIT ERROR has to be detected at that bit time, when the bit value that is monitored is different from the bit value that is sent. An exception is the sending of a 'recessive' bit during the stuffed bit stream of the ARBITRATION FIELD or during the ACK SLOT. Then no BIT ERROR occurs when a 'dominant' bit is monitored. A TRANSMITTER sending a PASSIVE ERROR FLAG and detecting a 'dominant' bit does not interpret this as a BIT ERROR.

● **BIT STUFF ERROR:**

A STUFF ERROR has to be detected at the bit time of the 6th consecutive equal bit level in a message field that should be coded by the method of bit stuffing.

● **FORM ERROR:**

A FORM ERROR has to be detected when a fixed-form bit field contains one or more illegal bits. (Exception is the detection of a dominant bit during the last bit of END OF FRAME by a RECEIVER, or the detection of a dominant bit during the last bit of ERROR DELIMITER or OVERLOAD DELIMITER by any node). FORM-ERROR.

● **ACK ERROR:**

An ACK ERROR has to be detected by a transmitter whenever it does not monitor a 'dominant' bit during the ACK SLOT.

● **CRC ERROR:**

The CRC sequence consists of the result of the CRC calculation by the transmitter. The receivers calculate the CRC in the same way as the transmitter. A CRC ERROR has to be detected, if the calculated result is not the same as that received in the CRC sequence.

29.4 Register Description

29.4.1 Internal Address Mapping

Slave address can be divided into different length for different usage, which is shown as

follows.

29.4.2 Registers Summary

Name	Offset	Size	Reset Value	Description
<u>CAN_MODE</u>	0x0000	W	0x00000000	Mode Configure Register
<u>CAN_CMD</u>	0x0004	W	0x00000000	Request Command Register
<u>CAN_STATE</u>	0x0008	W	0x00000000	Controller State Register
<u>CAN_INT</u>	0x000C	W	0x00000000	Interrupt State Register
<u>CAN_INT_MASK</u>	0x0010	W	0x00000000	Interrupt Mask Registers
<u>CAN_DMA_CTRL</u>	0x0014	W	0x00000000	DMA Mode Control Register
<u>CAN_BITTIMING</u>	0x0018	W	0x00000000	Bit Timing Configure Register. This register work in CAN mode.
<u>CAN_ARBITFAIL</u>	0x0028	W	0x00000000	Arbiter Fail Code Register
<u>CAN_ERROR_CODE</u>	0x002C	W	0x00000000	Error Code Register
<u>CAN_RXERRORCNT</u>	0x0034	W	0x00000000	Receive Error Counter Register
<u>CAN_TXERRORCNT</u>	0x0038	W	0x00000000	Transmit Error Counter Register
<u>CAN_IDCODE</u>	0x003C	W	0x00000000	ID Code Register
<u>CAN_IDMASK</u>	0x0040	W	0x00000000	ID Code Bit Mask Register
<u>CAN_TXFRAMEINFO</u>	0x0050	W	0x00000000	CAN Mode TX Frame Information Configuration Register
<u>CAN_TXID</u>	0x0054	W	0x00000000	CAN Mode Transmit ID Code Register.
<u>CAN_TXDATA0</u>	0x0058	W	0x00000000	CAN Mode Transmit Data0 Register.
<u>CAN_TXDATA1</u>	0x005C	W	0x00000000	CAN Mode Transmit Data1 Register.
<u>CAN_RXFRAMEINFO</u>	0x0060	W	0x00000000	CAN Mode RX Frame Information Register
<u>CAN_RXID</u>	0x0064	W	0x00000000	CAN Mode Receive ID.
<u>CAN_RXDATA0</u>	0x0068	W	0x00000000	CAN Mode Receive Data Reg0.
<u>CAN_RXDATA1</u>	0x006C	W	0x00000000	CAN Mode Receive Data Reg1.
<u>CAN_RTL_VERSION</u>	0x0070	W	0x00000022	CAN RTL version
<u>CAN_FD_NOMINAL_BITTIMING</u>	0x0100	W	0x00000000	CAN Nominal Bit Timing Configure Register
<u>CAN_TIMESTAMP_CTRL</u>	0x010C	W	0x00000000	Timestamp Counter Configure Register
<u>CAN_TIMESTAMP</u>	0x0110	W	0x00000000	Timestamp Counter
<u>CAN_TXEVENT_FIFO_CTRL</u>	0x0114	W	0x00000000	Tx Event FIFO Configure register
<u>CAN_RX_FIFO_CTRL</u>	0x0118	W	0x00000000	Rx FIFO Configure Register
<u>CAN_AFR_CTRL</u>	0x011C	W	0x00000000	Acceptance Filter Register Control
<u>CAN_IDCODE0</u>	0x0120	W	0x00000000	ID Code Register 0
<u>CAN_IDMASK0</u>	0x0124	W	0x00000000	ID Mask Register 0
<u>CAN_IDCODE1</u>	0x0128	W	0x00000000	ID Code Register 1
<u>CAN_IDMASK1</u>	0x012C	W	0x00000000	ID Mask Register 1

Name	Offset	Size	Reset Value	Description
<u>CAN_IDCODE2</u>	0x0130	W	0x00000000	ID Code Register 2
<u>CAN_IDMASK2</u>	0x0134	W	0x00000000	ID Mask Register 2
<u>CAN_IDCODE3</u>	0x0138	W	0x00000000	ID Code Register 3
<u>CAN_IDMASK3</u>	0x013C	W	0x00000000	ID Mask Register 3
<u>CAN_IDCODE4</u>	0x0140	W	0x00000000	ID Code Register 4
<u>CAN_IDMASK4</u>	0x0144	W	0x00000000	ID Mask Register 4
<u>CAN_NEW_TXFRAMEINFO</u>	0x0200	W	0x00000000	CAN TX Frame Information Configuration Register
<u>CAN_NEW_TXID</u>	0x0204	W	0x00000000	CAN Transmit ID Register
<u>CAN_NEW_TXDATA0</u>	0x0208	W	0x00000000	CAN Transmit DATA0
<u>CAN_NEW_TXDATA1</u>	0x020C	W	0x00000000	CAN Transmit DATA1
<u>CAN_NEW_RXFRAMEINFO</u>	0x0300	W	0x00000000	CAN RX Frame Information Configuration Register
<u>CAN_NEW_RXID</u>	0x0304	W	0x00000000	CAN Receive ID
<u>CAN_NEW_RXTIMESTAMP</u>	0x0308	W	0x00000000	CAN Receive Timestamp
<u>CAN_NEW_RXDATA0</u>	0x030C	W	0x00000000	CAN Receive Data Reg0.
<u>CAN_NEW_RXDATA1</u>	0x0310	W	0x00000000	CAN Receive Data Reg1.
<u>CAN_RX_FIFO_RDATA</u>	0x0400	W	0x00000000	Rx FIFO Read Data
<u>CAN_TXE_FIFO_RDATA</u>	0x0500	W	0x00000000	Tx Event FIFO Read Data

Notes: **Size:** **B**- Byte (8 bits) access, **HW**- Half WORD (16 bits) access, **W**-WORD (32 bits) access, **DW**- Double WORD (64 bits) access

29.4.3 Detail Registers Description

CAN_MODE

Address: Operational Base + offset (0x0000)

Bit	Attr	Reset Value	Description
31:13	RO	0x0000	reserved
12	RW	0x0	space_rx_mode Interframe Spacing RX Mode. 1'b0: Enable. 1'b1: Disable.
11	RW	0x0	auto_bus_on Auto Bus on Enable. 1'b0: After the RKCAN has entered bus_off state, the software can start a bus_off_recovery sequence by resetting the work_mode to 0. 1'b1: Automatic reset TEC/REC to bus_on after 128 occurrences of 11 consecutive recessive bits have been monitored on the bus.
10	RW	0x0	auto_retx_mode Auto Retransmission Mode. 1'b0: Disable. RKCAN return to idle state after transmit DATA/RTR frame failed, and set tx_req to 0. 1'b1: Enable. The RKCAN automatically retransmit frames which have lost arbitration or have been disturbed by errors during transmission.
9	RW	0x0	ovld_mode Overload Mode. 1'b0: overload lite mode. 1'b1: overload extended mode. RKCAN

Bit	Attr	Reset Value	Description
8	RW	0x0	cover_mode Rx Data Cover Mode. 1'b0: RKCAN can't receive new frame before receive registers cleared. 1'b1: RKCAN can receive new frame before receive registers cleared, and new rxdata will cover old rxdata.
7	RW	0x0	rxsort_mode RX Data Sort Mode. 1'b0: The data received first is placed in the lower address. 1'b1: The data received first moves to the upper address, and the data received later is placed at the lower address.
6	RW	0x0	txorder_mode TX Data Order Mode. 1'b0: tx_data1[7:0] --> tx_data1[15:8] --> tx_data1[23:16] --> tx_data1[31:24] --> tx_data2[7:0] --> tx_data2[15:8] --> tx_data2[23:16] --> tx_data2[31:24]. 1'b1: tx_data2[31:0] --> tx_data1[31:0].
5	RW	0x0	rxstx_mode Receive Self Transmit data mode. 1'b0: Disable; 1'b1: Enable. When the RKCAN sends data, it can also receive the data sent by itself.
4	RW	0x0	lback_mode Loopback Mode. 1'b0: Disable. 1'b1: Enable.
3	RW	0x0	silent_mode Silent Mode. 1'b0: Disable. 1'b1: Enable.
2	RW	0x0	self_test Self check ACK slot when TX frame. 1'b0: Normal 1'b1: Self-test mode. When in self_test mode, the receiver does not need to return an ACK signal when receiving data. Therefore, no error ack will occur in self_test mode. When in lback_mode or silent_mode, it needs to enable self_test mode when sending frame.
1	RW	0x0	sleep_mode Sleep Mode This is the Sleep mode request bit. 1'b0: No such request. 1'b1: Request core to be in Sleep mode. This bit is cleared when the core wakes up from Sleep mode.
0	RW	0x0	work_mode Work Mode. 1'b0: Idle mode, CAN stop transmit and registers can be configured. 1'b1: Work mode, CAN enter the working mode, Receive/transmit data.

CAN_CMD

Address: Operational Base + offset (0x0004)

Bit	Attr	Reset Value	Description
31:2	RO	0x00000000	reserved

Bit	Attr	Reset Value	Description
1	R/W SC	0x0	tx1_req Transmit request enable 1'b0: Disable 1'b1: Enable When 'tx1_req' is enable, the RKCAN buffer1 is in the transmit mode and cannot receive frames from other CANs. When the RKCAN buffer1 transmission frame complete, 'tx1_req' is automatically cleared to 0. Also, when RKCAN is set 'work_mode' to 'reset_mode', tx_req' is cleared to 0.
0	R/W SC	0x0	tx0_req Transmit request enable 1'b0: Disable 1'b1: Enable When 'tx0_req' is enable, the RKCAN buffer0 is in the transmit mode and cannot receive frames from other CANs. When the RKCAN buffer0 transmission frame complete, 'tx0_req' is automatically cleared to 0. Also, when RKCAN is set 'work_mode' to 'reset_mode', tx_req' is cleared to 0.

CAN_STATE

Address: Operational Base + offset (0x0008)

Bit	Attr	Reset Value	Description
31:7	RO	0x0000000	reserved
6	RO	0x0	sleep_state Sleep state 1'b0: None 1'b1: CAN controller is in the sleep state
5	RO	0x0	bus_off_state Bus off state 1'b0: None 1'b1: Bus off state: When the error counter is incremented to 255, the CAN controller will enter the bus off state, generate an error warning interrupt and enter reset mode, waiting for the CPU to restart. (rx/tx_err_cnt >=32'd255)
4	RO	0x0	error_warning_state Error warning state 1'b0: None 1'b1: Error state: At least one error counter has reached the error warning threshold (rx/tx_err_cnt >=32'd96)
3	RO	0x0	tx_period Transmit state 1'b0: Not in transmit state 1'b1: CAN controller is in the transmit state
2	RO	0x0	rx_period Receive state 1'b0: Not in receive state 1'b1: CAN controller is in the receive state
1	RO	0x0	tx_buffer_full Transmit buffer full flag bit 1'b0: Not full 1'b1: TX buffer is full. There is data in buffer waiting to be sent or being sent.

Bit	Attr	Reset Value	Description
0	RO	0x0	rx_buffer_full Receive buffer full flag bit 1'b0: Not full 1'b1: RX buffer is full. A complete message is stored in the buffer.

CAN_INT

Address: Operational Base + offset (0x000C)

Bit	Attr	Reset Value	Description
31:15	RO	0x00000	reserved
14	W1 C	0x0	wakeup_int Wakeup Interrupt 1'b0: None 1'b1: Indicates that the core entered Normal mode from Sleep mode. Write 1 then clear.
13	W1 C	0x0	txe_fifo_full_int TXE FIFO Full Interrupt 1'b0: None 1'b1: Indicates that TX Event FIFO is full based on watermark programming. The interrupt continues to assert as long as the TX Event FIFO Full Level is above TX Event FIFO Full watermark.
12	W1 C	0x0	tx_event_fifo_overflow_int TXE FIFO Overflow Interrupt 1'b0 None 1'b1 Indicates that a message has been lost. This condition occurs when the core has successfully transmitted a message for which an event store is requested but the TX Event FIFO is full.
11	W1 C	0x0	timestamp_counter_overflow_int Timestamp Overflow Interrupt 1'b0: None 1'b1: Indicates that Timestamp counter rolled over (from 0xffff to 0x0). Write 1 then clear.
10	W1 C	0x0	bus_off_recovery_int Bus off Recover Interrupt 1'b0: None 1'b1: Indicates that the core is recovered from Bus-off mode. Write 1 then clear.
9	W1 C	0x0	bus_off_int Bus Off Interrupt 1'b0: None 1'b1: Indicates that the CAN core entered Bus-off mode. Write 1 then clear.
8	W1 C	0x0	rx_fifo_overflow_int RX FIFO Overflow Interrupt (RX FIFO ENABLE). 1'b0: None 1'b1: Indicates that a message has been lost. This condition occurs when a new message with ID matching to Receive FIFO is received and the Receive FIFO is full.
7	W1 C	0x0	rx_fifo_full_int RX FIFO full Interrupt (FIFO Mode enable). 1'b0: None 1'b1: Indicates that RX FIFO is full based on watermark setting.

Bit	Attr	Reset Value	Description
6	W1 C	0x0	error_int Bus Error Interrupt 1'b0: None 1'b1: CAN bus error interrupt. This interrupt is generated when a bus error is detected. Error details can refer to the ERROR_CODE register. Write 1 then clear.
5	W1 C	0x0	tx_arbit_fail_int Transmit Arbitration loss Interrupt 1'b0: None 1'b1: Arbitration loss interrupt. This interrupt is generated when the loss of arbitration is turned into a receiver. Write 1 then clear.
4	W1 C	0x0	passive_error_int Passive Error Interrupt 1'b0: None 1'b1: Passive error interrupt. This interrupt is generated when the controller enters an error passive state (at least one error counter reaches 127) or returns from the error passive state to the error active state. Write 1 then clear.
3	W1 C	0x0	overload_int Overload Interrupt 1'b0: None 1'b1: CAN bus overload interrupt. This interrupt is generated when a overload frame is generated.
2	W1 C	0x0	error_warning_int Error warning interrupt 1'b0: None 1'b1: Error_warning_int. This interrupt is generated when the error_warning_state bits change. Write 1 then clear.
1	W1 C	0x0	tx_finish_int Transmit finish interrupt 1'b0: None 1'b1: Transmit finish interrupt. CAN controller sends the message. Write 1 then clear.
0	W1 C	0x0	rx_finish_int Receive Finish Interrupt 1'b0: None 1'b1: Receive finish int. CAN controller has received the message and the rx_buffer is full. Write 1 then clear.

CAN INT MASK

Address: Operational Base + offset (0x0010)

Bit	Attr	Reset Value	Description
31:15	RO	0x00000	reserved
14	RW	0x0	wakeup_int_mask 1'b0: Unmasked 1'b1: Masked
13	RW	0x0	txe_fifo_full_int_mask 1'b0: Unmasked 1'b1: Masked
12	RW	0x0	txe_fifo_overflow_int_mask 1'b0: Unmasked 1'b1: Masked
11	RW	0x0	timestamp_cnt_overflow_int_mask 1'b0: Unmasked 1'b1: Masked

Bit	Attr	Reset Value	Description
10	RW	0x0	bus_off_recovery_int_mask 1'b0: Unmasked 1'b1: Masked
9	RW	0x0	bus_off_int_mask 1'b0: Unmasked 1'b1: Masked
8	RW	0x0	rx_fifo_overflow_int_mask 1'b0: Unmasked 1'b1: Masked
7	RW	0x0	rx_fifo_full_int_mask 1'b0: Unmasked 1'b1: Masked
6	RW	0x0	error_int_mask 1'b0: Unmasked 1'b1: Masked
5	RW	0x0	tx_arbit_fail_int_mask 1'b0: Unmasked 1'b1: Masked
4	RW	0x0	passive_error_int_mask 1'b0: Unmasked 1'b1: Masked
3	RW	0x0	rx_buffer_overflow_int_mask 1'b0: Unmasked 1'b1: Masked
2	RW	0x0	error_warning_int_mask 1'b0: Unmasked 1'b1: Masked
1	RW	0x0	tx_finish_int_mask 1'b0: Unmasked 1'b1: Masked
0	RW	0x0	rx_finish_int_mask 1'b0: Unmasked 1'b1: Masked

CAN DMA CTRL

Address: Operational Base + offset (0x0014)

Bit	Attr	Reset Value	Description
31:2	RO	0x00000000	reserved
1	RW	0x0	dma_rx_mode DMA receive request enable 1'b0: Disable 1'b1: Enable When dma_rx_mode is enabled, the RKCAN is in the DMA receive mode. Also, when RKCAN is set work_mode to reset_mode, tx_req is clear.
0	RW	0x0	dma_tx_mode DMA Transmit request enable 1'b0: Disable 1'b1: Enable When dma_tx_mode is enabled, the RKCAN is in the DMA transmit mode. When RKCAN is set work_mode to reset_mode, tx_req is cleared to 0.

CAN BITTIMING

Address: Operational Base + offset (0x0018)

Bit	Attr	Reset Value	Description
31:17	RO	0x0000	reserved
16	RW	0x0	sample_mode CAN controller sampling mode configuration register. 1'b0: Single sample mode 1'b1: Three sample mode
15:14	RW	0x0	sjw SJW: Resynchronization Jump Width Each unit has a synchronization error due to a clock frequency deviation or a transmission delay. SJW is to compensate for the maximum value of this error.
13:8	RW	0x00	brp brp: system prescale coefficient $T_{sclk} = 2 \times T_{clk} \times (brp + 1)$.
7	RO	0x0	reserved
6:4	RW	0x0	tseg2 Phase buffer segment 2 $T_{phase_seg2} = T_{sclk} \times (tseg2 + 1)$
3:0	RW	0x0	tseg1 Phase buffer segment 1 $T_{phase_seg1} = T_{sclk} \times (tseg1 + 1)$

CAN ARBITFAIL

Address: Operational Base + offset (0x0028)

Bit	Attr	Reset Value	Description
31:7	RO	0x0000000	reserved
6:0	RO	0x00	arbit_fail_code This register indicates the bit position of arbitration section where the arbitration was lost.

CAN ERROR CODE

Address: Operational Base + offset (0x002C)

Bit	Attr	Reset Value	Description
31:30	RO	0x0	reserved
29	RO	0x0	error_phase 1'b0: Arbitration Phase 1'b1: Data Phase
28:26	RO	0x0	error_type Error type: 3'b000: BIT ERROR 3'b001: BIT STUFF ERROR 3'b010: FORM ERROR 3'b011: ACK ERROR 3'b100: CRC ERROR
25	RO	0x0	error_direction 1'b0: TX error 1'b1: RX error

Bit	Attr	Reset Value	Description
24:16	RO	0x000	tx_error_position Indicate transmit error position. 9'b000000001: TRANSMIT_IDLE. 9'b000000010: TRANSMIT_SOF_DLC. 9'b000000100: TRANSMIT_DATA. 9'b000001000: TRANSMIT_STUFF_COUNT. 9'b000010000: TRANSMIT_CRC. 9'b000100000: TRANSMIT_ACK_EOF. 9'b001000000: TRANSMIT_ACK. 9'b010000000: TRANSMIT_ERROR. 9'b100000000: TRANSMIT_OVERLOAD.
15:0	RO	0x0000	rx_error_position Indicate receive error position 16'b00000000000000001: RECEIVE_IDLE 16'b00000000000000010: RECEIVE_SOF_IDE 16'b00000000000000100: RECEIVE_ID2_RTR 16'b00000000000001000: RECEIVE_FDF 16'b00000000000010000: RECEIVE_RES 16'b0000000000100000: RECEIVE_BRS_ESI 16'b0000000001000000: RECEIVE_DLC 16'b0000000010000000: RECEIVE_DATA 16'b0000000100000000: RECEIVE_STUFF_COUNT 16'b0000001000000000: RECEIVE_CRC 16'b0000010000000000: RECEIVE_CRC_LIM 16'b0000100000000000: RECEIVE_ACK 16'b0001000000000000: RECEIVE_ACK_LIM 16'b0010000000000000: RECEIVE_EOF 16'b0100000000000000: RECEIVE_SPACE 16'b1000000000000000: RECEIVE_BUF_INT

CAN_RXERRORCNT

Address: Operational Base + offset (0x0034)

Bit	Attr	Reset Value	Description
31:8	RO	0x000000	reserved
7:0	RO	0x00	rx_err_cnt Receive Error Counter (REC). Actual state of the Receive Error Counter. Value between 0 and 127.

CAN_TXERRORCNT

Address: Operational Base + offset (0x0038)

Bit	Attr	Reset Value	Description
31:9	RO	0x000000	reserved
8:0	RO	0x000	tx_err_cnt Transmit Error Counter (TEC). Actual state of the Transmit Error Counter. Value between 0 and 255.

CAN_IDCODE

Address: Operational Base + offset (0x003C)

Bit	Attr	Reset Value	Description
31:29	RO	0x0	reserved

Bit	Attr	Reset Value	Description
28:0	RW	0x00000000	id_code CAN controller ID code: Standard frame ID: id_code[10:0]; Extended frame ID: id_code[28:0].

CAN_IDMASK

Address: Operational Base + offset (0x0040)

Bit	Attr	Reset Value	Description
31:29	RO	0x0	reserved
28:0	RW	0x00000000	id_mask ID mask for each ID code bit 1'b0: Unmasked 1'b1: Masked

CAN_TXFRAMEINFO

Address: Operational Base + offset (0x0050)

Bit	Attr	Reset Value	Description
31:8	RO	0x000000	reserved
7	RW	0x0	txframe_format 1'b0: Standard frame 1'b1: Extended frame
6	RW	0x0	tx_rtr 1'b0: Data frame 1'b1: Remote frame
5:4	RO	0x0	reserved
3:0	RW	0x0	txdata_length Transmit data length configure register. (unit: byte) 4'b0000: 0byte 4'b0001: 1byte 4'b0010: 2byte 4'b0011: 3byte 4'b0100: 4byte 4'b0101: 5byte 4'b0110: 6byte 4'b0111: 7byte 4'b1000: 8byte others: reserved

CAN_TXID

Address: Operational Base + offset (0x0054)

Bit	Attr	Reset Value	Description
31:29	RO	0x0	reserved
28:0	RW	0x00000000	tx_id can_tx_id[28:0]

CAN_TXDATA0

Address: Operational Base + offset (0x0058)

Bit	Attr	Reset Value	Description
31:0	RW	0x00000000	tx_data0 tx_data0

CAN_TXDATA1

Address: Operational Base + offset (0x005C)

Bit	Attr	Reset Value	Description
31:0	RW	0x00000000	tx_data1 tx_data1

CAN_RXFRAMEINFO

Address: Operational Base + offset (0x0060)

Bit	Attr	Reset Value	Description
31:8	RO	0x000000	reserved
7	RC	0x0	rxframe_format 1'b0: Standard frame 1'b1: Extended frame This register needs to be read(clear) before receiving the next frame.
6	RC	0x0	rx_rtr 1'b0: Data frame 1'b1: Remote frame This register needs to be read(clear) before receiving the next frame.
5:4	RO	0x0	reserved
3:0	RC	0x0	rxdata_length Receive data length register. (unit: byte) 4'b0000: 0byte 4'b0001: 1byte 4'b0010: 2byte 4'b0011: 3byte 4'b0100: 4byte 4'b0101: 5byte 4'b0110: 6byte 4'b0111: 7byte 4'b1000: 8byte others: reserved This register needs to be read(clear) before receiving the next frame.

CAN_RXID

Address: Operational Base + offset (0x0064)

Bit	Attr	Reset Value	Description
31:29	RO	0x0	reserved
28:0	RC	0x00000000	rx_id rx_id[28:0] This register needs to be read(clear) before receiving the next frame.

CAN_RXDATA0

Address: Operational Base + offset (0x0068)

Bit	Attr	Reset Value	Description
31:0	RC	0x00000000	rx_data0 rx_data0[31:0] This register needs to be read(clear) before receiving the next frame.

CAN_RXDATA1

Address: Operational Base + offset (0x006C)

Bit	Attr	Reset Value	Description
31:0	RC	0x00000000	rx_data1 rx_data1[31:0] This register needs to be read(clear) before receiving the next frame.

CAN RTL VERSION

Address: Operational Base + offset (0x0070)

Bit	Attr	Reset Value	Description
31:0	RO	0x00000022	version CAN rtl version = 32'h22

CAN FD NOMINAL BITTIMING

Address: Operational Base + offset (0x0100)

Bit	Attr	Reset Value	Description
31	RW	0x0	sample_mode CAN controller sampling mode configuration register. 1'b0: Single sample mode 1'b1: Three sample mode
30:24	RW	0x00	sjw SJW: Resynchronization Jump Width Each unit has a synchronization error due to a clock frequency deviation or a transmission delay. SJW is to compensate for the maximum value of this error.
23:16	RW	0x00	brq brp: system prescale coefficient $T_{sclk} = 2 \times T_{clk} \times (brp + 1)$.
15	RO	0x0	reserved
14:8	RW	0x00	tseg2 Phase buffer segment 2 $T_{phase_seg2} = T_{sclk} \times (tseg2 + 1)$
7:0	RW	0x00	tseg1 Phase buffer segment 1 $T_{phase_seg1} = T_{sclk} \times (tseg1 + 1)$

CAN FD DATA BITTIMING

Address: Operational Base + offset (0x0104)

Bit	Attr	Reset Value	Description
31:22	RO	0x000	reserved
21	RW	0x0	sample_mode CAN controller sampling mode configuration register. 1'b0: Single sample mode 1'b1: Three sample mode
20:17	RW	0x0	sjw SJW: Resynchronization Jump Width Each unit has a synchronization error due to a clock frequency deviation or a transmission delay. SJW is to compensate for the maximum value of this error.
16:9	RW	0x00	brq brp: system prescale coefficient $T_{sclk} = 2 \times T_{clk} \times (brp + 1)$.
8:5	RW	0x0	tseg2 Phase buffer segment 2 $T_{phase_seg2} = T_{sclk} \times (tseg2 + 1)$

Bit	Attr	Reset Value	Description
4:0	RW	0x00	tseg1 Phase buffer segment 1 $T_{phase_seg1} = T_{clk} \times (tseg1 + 1)$

CAN TRANSMIT DELAY COMPENSATION

Address: Operational Base + offset (0x0108)

Bit	Attr	Reset Value	Description
31:7	RO	0x0000000	reserved
6:1	RW	0x00	tdc_offset Transmitter Delay Compensation Offset This offset is specified in CAN clock cycles and is added to the measured transmitter delay to place the Secondary Sample Point (SSP) at appropriate position (for example, set this to half data bit time in terms of CAN clock cycles to place SSP in the middle of the data bit).
0	RW	0x0	tdc_enable Transmitter Delay Compensation (TDC) Enable 1'b0: TDC is disabled 1'b1: Enables TDC function as specified in the CAN FD standard

CAN TIMESTAMP CTRL

Address: Operational Base + offset (0x010C)

Bit	Attr	Reset Value	Description
31:7	RO	0x0000000	reserved
6:1	RW	0x00	time_base_counter_prescale Time Base Counter Prescaler bit 0: TBC increments every 1 clock 1: TBC increments every 2 clocks ... 32: TBC increments every 32 clocks
0	RW	0x0	time_base_counter_enable Time Base Counter (TBC) Enable 1'b0: Disable and reset TBC 1'b1: Enables TBC

CAN TIMESTAMP

Address: Operational Base + offset (0x0110)

Bit	Attr	Reset Value	Description
31:0	RW	0x00000000	time_base_counter This Status field gives running value of the timestamp counter.

CAN TXEVENT FIFO CTRL

Address: Operational Base + offset (0x0114)

Bit	Attr	Reset Value	Description
31:9	RO	0x0000000	reserved
8:5	RO	0x0	txe_fifo_cnt TXE_FIFO_FRAME_CNT This field represents number of frames in TXE FIFO
4:1	RW	0x0	txe_fifo_watermark TX Event FIFO generates FULL interrupt based on the value programmed in this field. Set it within (1-15) range. The TX FIFO Full Watermark interrupt in the ISR register continues to assert as long as the TX Event FIFO Fill Level is above TX Event FIFO Full watermark.

Bit	Attr	Reset Value	Description
0	RW	0x0	txe_fifo_enable transmit event FIFO enable 1'b0: Disable 1'b1: Enable

CAN RX FIFO CTRL

Address: Operational Base + offset (0x0118)

Bit	Attr	Reset Value	Description
31:8	RO	0x0000000	reserved
7:5	RO	0x0	rx_fifo_cnt RX_FIFO_FRAME_CNT This field represents number of frames in Rx FIFO
6	RO	0x0	reserved
3:1	RW	0x0	rx_fifo_full_watermark RX FIFO Full Watermark. RX FIFO generates FULL interrupt based on the value programmed in this field. Set it within (1-5) range. The RX FIFO Full Watermark interrupt in the ISR register continues to assert as long as the RX FIFO-0 Fill Level is above RX FIFO Full watermark. This field can be written to only when CEN bit in SRR is 0.
0	RW	0x0	rx_fifo_enable receive FIFO enable bit 1'b0: Disable 1'b1: Enable

CAN AFR CTRL

Address: Operational Base + offset (0x011C)

Bit	Attr	Reset Value	Description
31:5	RO	0x0000000	reserved
4	RW	0x0	UAF5 Enable the use of acceptance filter pair 5 1'b0: Disable 1'b1: Enable
3	RW	0x0	UAF4 Enable the use of acceptance filter pair 4 1'b0: Disable 1'b1: Enable
2	RW	0x0	UAF3 Enable the use of acceptance filter pair 3 1'b0: Disable 1'b1: Enable
1	RW	0x0	UAF2 Enable the use of acceptance filter pair 2 1'b0: Disable 1'b1: Enable
0	RW	0x0	UAF1 Enable the use of acceptance filter pair 1 1'b0: Disable 1'b1: Enable

CAN IDCODE0

Address: Operational Base + offset (0x0120)

Bit	Attr	Reset Value	Description
31:29	RO	0x0	reserved
28:0	RW	0x00000000	id_code CAN controller ID code: Standard frame ID: id_code[10:0]; Extended frame ID: id_code[28:0].

CAN_IDMASK0

Address: Operational Base + offset (0x0124)

Bit	Attr	Reset Value	Description
31:29	RO	0x0	reserved
28:0	RW	0x00000000	id_mask ID mask for each bit of ID code 1'b0: Unmasked 1'b1: Masked

CAN_IDCODE1

Address: Operational Base + offset (0x0128)

Bit	Attr	Reset Value	Description
31:29	RO	0x0	reserved
28:0	RW	0x00000000	id_code CAN controller ID code: Standard frame ID: id_code[10:0]; Extended frame ID: id_code[28:0].

CAN_IDMASK1

Address: Operational Base + offset (0x012C)

Bit	Attr	Reset Value	Description
31:29	RO	0x0	reserved
28:0	RW	0x00000000	id_mask ID mask for each bit of ID code 1'b0: Unmasked 1'b1: Masked

CAN_IDCODE2

Address: Operational Base + offset (0x0130)

Bit	Attr	Reset Value	Description
31:29	RO	0x0	reserved
28:0	RW	0x00000000	id_code CAN controller ID code: Standard frame ID: id_code[10:0]; Extended frame ID: id_code[28:0].

CAN_IDMASK2

Address: Operational Base + offset (0x0134)

Bit	Attr	Reset Value	Description
31:29	RO	0x0	reserved
28:0	RW	0x00000000	id_mask ID mask for each bit of ID code 1'b0: Unmasked 1'b1: Masked

CAN_IDCODE3

Address: Operational Base + offset (0x0138)

Bit	Attr	Reset Value	Description
31:29	RO	0x0	reserved
28:0	RW	0x00000000	id_code CAN controller ID code: Standard frame ID: id_code[10:0]; Extended frame ID: id_code[28:0].

CAN_IDMASK3

Address: Operational Base + offset (0x013C)

Bit	Attr	Reset Value	Description
31:29	RO	0x0	reserved
28:0	RW	0x00000000	id_mask ID mask for each bit of ID code 1'b0: Unmasked 1'b1: Masked

CAN_IDCODE4

Address: Operational Base + offset (0x0140)

Bit	Attr	Reset Value	Description
31:29	RO	0x0	reserved
28:0	RW	0x00000000	id_code CAN controller ID code: Standard frame ID: id_code[10:0]; Extended frame ID: id_code[28:0].

CAN_IDMASK4

Address: Operational Base + offset (0x0144)

Bit	Attr	Reset Value	Description
31:29	RO	0x0	reserved
28:0	RW	0x00000000	id_mask ID mask for each bit of ID code 1'b0: Unmasked 1'b1: Masked

CAN_NEW_TXFRAMEINFO

Address: Operational Base + offset (0x0200)

Bit	Attr	Reset Value	Description
31:8	RO	0x000000	reserved
7	RW	0x0	txframe_format 1'b0: Standard frame 1'b1: Extended frame
6	RW	0x0	tx_rtr 1'b0: Data frame 1'b1: Remote frame
5	RW	0x0	reserved
4	RW	0x0	reserved

Bit	Attr	Reset Value	Description
3:0	RW	0x0	txdata_length Transmit data length configure register. (unit: byte) 4'b0000: 0byte 4'b0001: 1byte 4'b0010: 2byte 4'b0011: 3byte 4'b0100: 4byte 4'b0101: 5byte 4'b0110: 6byte 4'b0111: 7byte 4'b1000: 8byte 4'b1001: 12byte 4'b1010: 16byte 4'b1011: 20byte 4'b1100: 24byte 4'b1101: 32byte 4'b1110: 48byte 4'b1111: 64byte

CAN_NEW_TXID

Address: Operational Base + offset (0x0204)

Bit	Attr	Reset Value	Description
31:29	RO	0x0	reserved
28:0	RW	0x00000000	tx_id can_tx_id[28:0]

CAN_NEW_TXDATA0

Address: Operational Base + offset (0x0208)

Bit	Attr	Reset Value	Description
31:0	RW	0x00000000	tx_data0 tx_data0

CAN_NEW_TXDATA1

Address: Operational Base + offset (0x020C)

Bit	Attr	Reset Value	Description
31:0	RW	0x00000000	tx_data1 tx_data1

CAN_NEW_RXFRAMEINFO

Address: Operational Base + offset (0x0300)

Bit	Attr	Reset Value	Description
31:8	RO	0x000000	reserved
7	RO	0x0	rxframe_format 1'b0: Standard frame 1'b1: Extended frame
6	RO	0x0	rx_rtr 1'b0: Data frame 1'b1: Remote frame
5	RO	0x0	reserved
4	RO	0x0	reserved

Bit	Attr	Reset Value	Description
3:0	RO	0x0	rxdata_length Transmit data length configure register. (unit: byte) 4'b0000: 0byte 4'b0001: 1byte 4'b0010: 2byte 4'b0011: 3byte 4'b0100: 4byte 4'b0101: 5byte 4'b0110: 6byte 4'b0111: 7byte 4'b1000: 8byte 4'b1001: 12byte 4'b1010: 16byte 4'b1011: 20byte 4'b1100: 24byte 4'b1101: 32byte 4'b1110: 48byte 4'b1111: 64byte

CAN_NEW_RXID

Address: Operational Base + offset (0x0304)

Bit	Attr	Reset Value	Description
31:29	RO	0x0	reserved
28:0	RO	0x00000000	rx_id can_rx_id[28:0]

CAN_RXTIMESTAMP

Address: Operational Base + offset (0x0308)

Bit	Attr	Reset Value	Description
31:0	RO	0x00000000	timestamp rx_timestamp

CAN_NEW_RXDATA0

Address: Operational Base + offset (0x030C)

Bit	Attr	Reset Value	Description
31:0	RO	0x00000000	rx_data0 rx_data0[31:0] This register refresh after receiving the next frame.

CAN_NEW_RXDATA1

Address: Operational Base + offset (0x0310)

Bit	Attr	Reset Value	Description
31:0	RO	0x00000000	rx_data1 rx_data1[31:0] This register refresh after receiving the next frame.

CAN_RX_FIFO_RDATA

Address: Operational Base + offset (0x0400)

Bit	Attr	Reset Value	Description
31:0	RO	0x00000000	rx_fifo_rdata Receive FIFO read data value

CAN_TXE_FIFO_RDATA

Address: Operational Base + offset (0x0500)

Bit	Attr	Reset Value	Description
31:0	RO	0x00000000	txe_fifo_rdata Transmit event FIFO read data value

29.5 Interface Description

Table 29-3 CAN Interface Description

Module Pin	Direction	Pad Name	IOMUX Setting
CAN0mux0 Interface			
can0_rx	I	I2S1_SCLK_RX_M1/PDM0_CLK0_M1/PWM1_M0/I2C2_SDA_M0/CAN0_RX_M0/SPI0_MOSI_M0/PCIE30X1_0_CLKREQN_M0/GPIO0_C0_d	BUS_IOC_GPIO0C_IOMUX_SEL_L[3:0]=4'hb
can0_tx	O	I2S1_LRCK_TX_M1/PWM0_M0/I2C2_SCL_M0/CAN0_TX_M0/SPI0_CS1_M0/PCIE30X1_1_PERSTN_M0/GPIO0_B7_d	BUS_IOC_GPIO0B_IOMUX_SEL_H[15:12]=4'hb
CAN0mux1 Interface			
can0_rx	I	SDMMC_CLK/PDM1_CLK0_M0/TEST_CLKOUT_M0/MCU_JTAG_TMS_M0/CAN0_RX_M1/UART5_TX_M0/GPIO4_D5_d	BUS_IOC_GPIO4D_IOMUX_SEL_H[7:4]=4'h9
can0_tx	O	SDMMC_CMD/PDM1_CLK1_M0/MCU_JTAG_TCK_M0/CAN0_TX_M1/UART5_RX_M0/PWM7_IR_M1/GPIO4_D4_u	BUS_IOC_GPIO4D_IOMUX_SEL_H[3:0]=4'h9
CAN1mux0 Interface			
can1_rx	I	GMAC1_TXEN/I2S2_SCLK_TX_M1/CAN1_RX_M0/UART3_TX_M1/PWM12_M0/GPIO3_B5_u	BUS_IOC_GPIO3B_IOMUX_SEL_H[7:4]=4'h9
can1_tx	O	GMAC1_MCLKINOUT/I2S2_LRCK_TX_M1/CAN1_TX_M0/UART3_RX_M1/PWM13_M0/GPIO3_B6_d	BUS_IOC_GPIO3B_IOMUX_SEL_H[11:8]=4'h9
CAN1mux1 Interface			
can1_rx	I	CIF_HREF/BT1120_D8/I2S1_SDO1_M0/PCIE30X1_1_BUTTON_RSTN/I2C7_SCL_M3/UART8_RTSN_M0/PWM14_M1/SPI0_CS0_M1/CAN1_RX_M1/GPIO4_B2_u	BUS_IOC_GPIO4B_IOMUX_SEL_L[11:8]=4'hc
can1_tx	O	CIF_VSYNC/BT1120_D9/I2S1_SDO2_M0/PCIE20X1_2_BUTTON_RSTN/I2C7_SDA_M3/UART8_CTSN_M0/PWM15_IR_M1/CAN1_TX_M1/GPIO4_B3_u	BUS_IOC_GPIO4B_IOMUX_SEL_L[15:12]=4'hc
CAN2mux0 Interface			
can2_rx	I	CIF_D8/FSPI_CS0N_M2/PCIE30X4_CLKREQN_M2/HDMI_TX1_CEC_M2/CAN2_RX_M0/UART5_TX_M1/SPI3_CS0_M3/GPIO3_C4_u	BUS_IOC_GPIO3C_IOMUX_SEL_H[3:0]=4'h9

can2_tx	O	CIF_D9/FSPI_CS1N_M2/PCIE30X4_WAKE_N_M2/HDMI_TX1_SDA_M1/CAN2_TX_M0/UART5_RX_M1/SPI3_CS1_M3/GPIO3_C5_u	BUS_IOC_GPIO3C_IOMUX_SEL_H[7:4]=4'h9
CAN2mux1 Interface			
can2_rx	I	I2S1_SDO2_M1/PDM0_SDI2_M1/PWM3_IR_M0/I2C1_SCL_M2/CAN2_RX_M1/HDMI_TX0_SDA_M1/SPI3_CS0_M2/PCIE30X2_PERSTN_M0/SATA_CPDET/GPIO0_D4_u	BUS_IOC_GPIO0D_IOMUX_SEL_H[3:0]=4'ha
can2_tx	O	I2S1_SDO3_M1/CPU_BIG1_AVS/I2C1_SDA_M2/CAN2_TX_M1/HDMI_TX0_SCL_M1/SPI3_CS1_M2/SATA_MP_SWITCH/GPIO0_D5_u	BUS_IOC_GPIO0D_IOMUX_SEL_H[7:4]=4'ha

29.6 Application Notes

29.6.1 Controller initialization flow

The controller must configure the registers after power-up or hardware reset. During the operation of the controller, a software reset request may be sent and reconfigured (re-initialized) as shown below.

After the initialization is completed, the controller enters the working mode, sends the frame to be sent to the buffer, and then sets the "send request" flag of the command register to start transmitting.

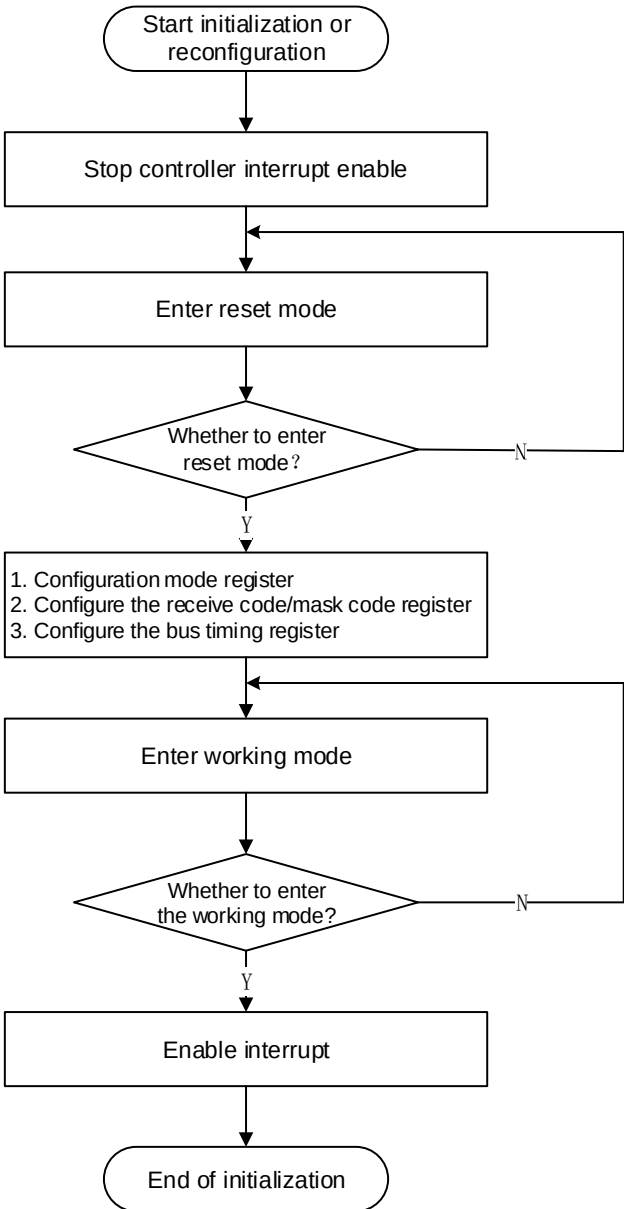


Fig 1-1 CAN Initialization Flow

29.6.2 Loop-back Mode

The controller must configure the registers after power-up or hardware reset. During the operation of the controller, a software reset request may be sent and reconfigured (re-initialized) as shown below.

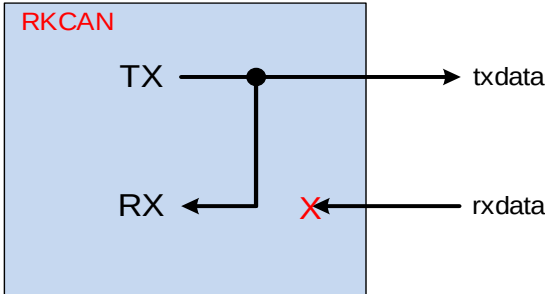


Fig 1-2 Loopback Mode

29.6.3 Silent Mode

The controller must configure the registers after power-up or hardware reset. During the operation of the controller, a software reset request may be sent and reconfigured (re-initialized) as shown below.

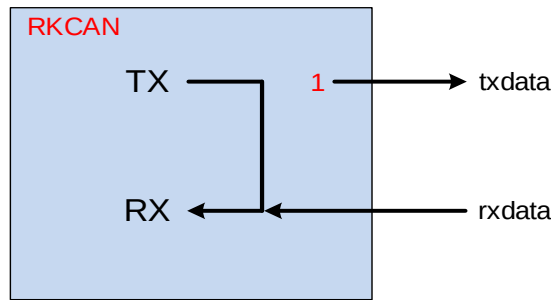


Fig 1-3 Silent Mode

29.6.4 Rxstx Mode

- 1'b0: RKCAN cannot receive the data sent by itself;
- 1'b1: RKCAN can receive the data sent by itself, and when the ID of the frame is the same as the ID_CODE of RKCAN itself, RKCAN will store the frame information into CAN_RXDATA1/2.

29.6.5 Txorder Mode

The Txorder Mode is used to determine the order in which data is sent.

- 1'b0: Data transmission order is "tx_data1[7:0] --> tx_data1[15:8] --> tx_data1[23:16] --> tx_data1[31:24] --> tx_data2[7:0] --> tx_data2[15:8] --> tx_data2[23:16] --> tx_data2[31:24]";
- 1'b1: Data transmission order is "tx_data2[31:0] --> tx_data1[31:0]".

29.6.6 Rxsort Mode

The Rxsort Mode is used to determine how the received data is stored.

- 1'b0: The data received first is placed in the lower address;
- 1'b1: The data received first moves to the upper address, and the data received later is placed at the lower address.

29.6.7 Cover Mode

- 1'b0: RKCAN can't receive new frame before receive registers cleared.
- 1'b1: RKCAN can receive new frame before receive registers cleared, and new RXDATA will cover old RXDATA.

29.6.8 Overload Mode

According to the CAN protocol, a CAN transceiver will generate an OVERLOAD frame in three cases:

- When the CAN is used as a receiver, if it is necessary to delay the next frame.
- Detection of a dominant bit at the first and second bit of Intermission.
- If an CAN node samples a dominant bit at the eight bits (the last bit) of an ERROR DELIMITER or OVERLOAD DELIMITER.
1'b0: Overload Lite Mode: RKCAN only generates overload frames in the CASE2.
1'b1: Overload Extended Mode: RKCAN can generates overload frames in all three cases.

29.6.9 Auto Retx Mode

Auto Retransmission Mode.

- 1'b0: Disable. RKCAN return to idle state after transmit DATA/RTR frame failed, and set tx_req to 0.
- 1'b1: Enable. The RKCAN automatically retransmit frames which have lost arbitration or have been disturbed by errors during transmission.

29.6.10 Auto Bus On

Auto Bus on Enable.

- 1'b0: After the RKCAN has entered bus_off state, the software can start a bus-off recovery sequence by resetting the work_mode to 0.
- 1'b1: Automatic reset TEC/REC to bus_on after 128 occurrences of 11 consecutive recessive bits have been monitored on the bus.

29.6.11 Sleep Mode

The core enters Sleep mode from Configuration or Normal mode when the SLEEP bit is 1 in the CAN_MODE register, the CAN bus is idle, and there are no pending transmission

requests.

The core enters Configuration mode when any configuration condition is satisfied. The core enters Normal mode (clearing the SLEEP request bit in the sleep register and also clearing the corresponding status bit) under the following (wake-up) conditions:

- Whenever the SLEEP bit is set to 0.
- Whenever the SLEEP bit is 1, and bus activity is detected.
- Whenever there is a new message for transmission.

Interrupts are generated when the core enters Sleep mode or wakes up from Sleep mode.

29.6.12 DMA Support

The CAN supports DMA signaling with the use of two output signals (dma_tx_mode and dma_rx_mode) to indicate when transmit buffer is empty or when the receive FIFO is full. The controller uses two DMA channels, one for the transmit data and one for the receive data.

DMA transmit and receive support enabled by setting corresponding bit in CAN_DMA_CTRL register. Once enabled whenever both transmit buffer is empty dma_tx_mode is set, DMA should write transmit message. At last CPU set transmit request bit to initiate CAN.

- If RX FIFO is disabled, dma_rx_mode is set when message received successfully, DMA should read RXDATA from receive registers.
- If RX FIFO is enabled, dma_rx_mode is set when message received and FIFO reached watermark, DMA should read RXDATA from Rx FIFO entrance.